

UPLA - EPISC

Escuela Profesional de Ingenieria de Sistemas y Computacion

Documentacion Tecnica del Proyecto

Arquitectura de software con Spring Boot, microservicios, Docker y MySQL

Proyecto	Sistema de Certificados de Talleres Tecnicos - EPISC
Arquitectura	Spring Boot + Microservicios + API Gateway + Docker
Base de datos	MySQL 8.4 en Docker, con schemas independientes por microservicio
Fecha	Julio de 2026

1. Resumen Ejecutivo

El proyecto implementa un sistema web para la gestión de certificados de talleres técnicos de EPISC. La solución se construye con arquitectura de microservicios, donde cada responsabilidad principal del negocio se separa en un servicio Spring Boot independiente y se integra mediante un API Gateway. El frontend es HTML, CSS y JavaScript; puede publicarse en Firebase Hosting, mientras el backend se ejecuta con Docker Compose en una PC, VPS o máquina virtual de Google Cloud.

2. Alineación con la Rubrica

La rúbrica revisada evalúa cinco dimensiones: análisis, diseño, implementación, implantación y sustentación. La siguiente matriz resume como el proyecto aporta evidencias para cada criterio.

Dimensión	Lo que pide la rúbrica	Evidencia en el proyecto
Análisis	Actores, requerimientos funcionales/no funcionales, reglas de negocio y trazabilidad.	Actores: alumno, coordinador y secretaria. Casos principales: registrar solicitud, subir informe, consultar estado, revisar trámite, emitir certificado y gestionar horario institucional.
Diseño	Dominios, microservicios, APIs REST, seguridad y persistencia independiente.	Servicios separados: autenticación, solicitudes, trámites, certificados, reportes y gateway. Cada servicio tiene responsabilidad delimitada y rutas REST documentadas.
Implementación	Microservicios independientes, REST, persistencia, seguridad y pruebas funcionales.	Spring Boot 3, Spring Data JPA, JWT, MySQL, endpoints REST y pruebas con curl, navegador y Docker.
Implantación	Docker, configuración de entorno, logs, manual técnico y evidencia funcional.	Docker Compose levanta MySQL, gateway y microservicios. Se despliega backend en VM de Google Cloud y frontend en Firebase Hosting.
Sustentación	Claridad técnica, justificación de decisiones y demostración funcional.	Este documento incluye guion de exposición, arquitectura, comandos de verificación y respuestas a preguntas probables.

3. Objetivos

- Gestionar solicitudes de certificados de talleres técnicos desde el registro hasta la emisión.
- Separar responsabilidades del negocio mediante microservicios Spring Boot.
- Usar Docker Compose para ejecutar todo el backend con una configuración reproducible.
- Mantener separación lógica de datos por servicio usando schemas independientes en MySQL.
- Permitir que el coordinador administre el horario de atención y habilite solicitudes manualmente si es necesario.

4. Actores y Casos de Uso

Actor	Funciones principales
Alumno	Iniciar sesión, registrar solicitud de certificado, adjuntar informe PDF, consultar estado del trámite y revisar certificados emitidos.
Coordinador	Revisar solicitudes, validar informe, avanzar etapas del trámite, gestionar horario de atención y permitir solicitudes fuera del horario normal cuando corresponda.
Secretaria	Consultar trámites listos, emitir certificados y mantener evidencia administrativa del proceso.

5. Requerimientos Funcionales

Código	Requerimiento	Módulo responsable
CU01	Registrar solicitud de certificado técnico.	ms-solicitudes
CU03	Consultar estado del trámite por código.	ms-solicitudes / ms-tramites

CU04	Subir informe tecnico en PDF.	ms-solicitudes
CU05	Revisar solicitudes e informes.	ms-tramites
CU06	Programar o registrar avance de sustentacion.	ms-tramites
CU07	Emitir certificado final.	ms-certificados
RF-HOR	Cambiar horario institucional y habilitar manualmente la atencion.	ms-solicitudes

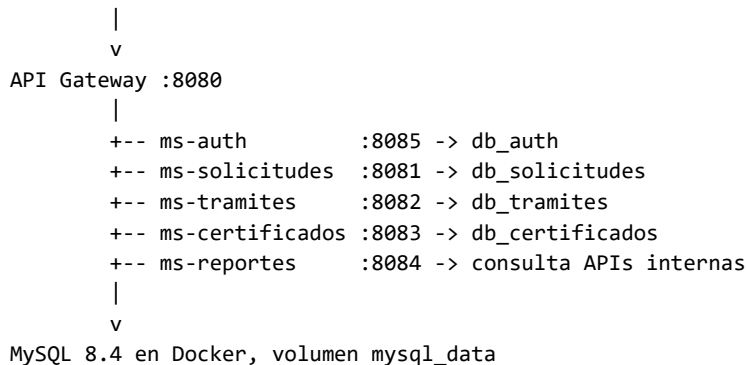
6. Requerimientos No Funcionales

Atributo	Decision implementada
Seguridad	Autenticacion con JWT; el API Gateway valida token para rutas protegidas.
Disponibilidad	Contenedores con politica restart: unless-stopped.
Portabilidad	Docker Compose permite ejecutar el sistema en Windows, Linux o VM en la nube.
Mantenibilidad	Servicios separados por dominio y paquetes Java independientes.
Escalabilidad	Cada microservicio puede escalarse o migrarse a infraestructura separada.
Observabilidad	Logs por contenedor con docker logs y actuador en el gateway.

7. Arquitectura General

El cliente no consume directamente los microservicios; todas las peticiones pasan por el API Gateway, que enruta segun el prefijo de la URL.

Frontend Firebase / Navegador



8. Microservicios

Servicio	Puerto	Responsabilidad	Persistencia
api-gateway	8080	Punto unico de entrada, CORS, validacion JWT y enrutamiento.	No usa schema propio.
ms-auth	8085	Login, usuarios demo, roles y generacion/validacion de JWT.	db_auth
ms-solicitudes	8081	Registro de solicitudes, talleres, informes PDF, estadisticas y horario de atencion.	db_solicitudes
ms-tramites	8082	Avance de etapas e historial del tramite.	db_tramites
ms-certificados	8083	Emission y descarga de certificados.	db_certificados
ms-reportes	8084	Reportes consolidados consumiendo APIs internas.	Consulta servicios internos.

9. Persistencia y Base de Datos

En una arquitectura de microservicios ideal, cada microservicio tiene una base de datos físicamente independiente. Para esta entrega universitaria se usó una sola instancia MySQL con schemas separados. La decisión conserva independencia lógica, reduce el costo operativo y permite demostrar el principio de separación de datos sin complicar el despliegue.

Schema	Servicio propietario	Contenido esperado
db_auth	ms-auth	Usuarios, credenciales, roles y estado activo.
db_solicitudes	ms-solicitudes	Solicitudes, talleres, horario de atención y archivos asociados.
db_tramites	ms-tramites	Historial y avance de etapas del trámite.
db_certificados	ms-certificados	Certificados emitidos y datos de descarga.

10. Seguridad

- El login se realiza en ms-auth mediante `/api/v1/auth/login`.
- El token JWT contiene correo, rol y expiración de sesión.
- El gateway permite rutas públicas como login y exige token en rutas protegidas.
- Los roles usados en la demo son ESTUDIANTE, COORDINADOR y SECRETARIA.

11. APIs REST Principales

Servicio	Endpoint	Uso
ms-auth	POST <code>/api/v1/auth/login</code>	Autenticar usuario y devolver token JWT.
ms-auth	GET <code>/api/v1/auth/usuarios</code>	Listar usuarios registrados.
ms-solicitudes	GET <code>/api/v1/talleres</code>	Listar talleres técnicos disponibles.
ms-solicitudes	POST <code>/api/v1/solicitudes</code>	Registrar solicitud de certificado.
ms-solicitudes	GET <code>/api/v1/solicitudes/codigo/{codigo}</code>	Consultar solicitud por código de trámite.
ms-solicitudes	GET/PUT <code>/api/v1/horario</code>	Consultar o actualizar horario institucional.
ms-solicitudes	GET <code>/api/v1/horario/estado</code>	Saber si el sistema está abierto para registrar solicitudes.
ms-tramites	POST <code>/api/v1/tramites/avanzar-etapa</code>	Avanzar etapa del trámite.
ms-certificados	POST <code>/api/v1/certificados/emitir</code>	Emitir certificado.
ms-reportes	GET <code>/api/v1/reportes</code>	Obtener resumen consolidado.

12. Despliegue

El sistema se puede ejecutar localmente o en una VM. En la entrega se trabajó con Docker y una VM de Google Cloud.

```
cd "C:\Users\diego\Music\proyecto-episc-certificados\proyecto-episc-certificados"
docker compose up -d --build
docker ps
```

Frontend publicado en Firebase Hosting:

<https://episc-certificados.web.app>

Backend en Google Cloud VM:

VM: episc-backend | Zona: us-central1-a | Puerto principal: 8080

13. Evidencias de Funcionamiento

- docker ps muestra api-gateway, ms-auth, ms-solicitudes, ms-tramites, ms-certificados, ms-reportes y episc-mysql en estado Up.
- Login con alumno@gmail.com, coordinador@gmail.com y secretaria@gmail.com usando clave 123456.
- MySQL muestra schemas db_auth, db_solicitudes, db_tramites y db_certificados.
- El endpoint /api/v1/horario/estado devuelve si la atencion esta abierta y si fue habilitada manualmente por coordinacion.

14. Decision Tecnica: Horario Configurable

El sistema originalmente bloqueaba el registro de solicitudes fuera del horario institucional. Se agrego una mejora para que el coordinador pueda cambiar el horario y, ademas, activar una apertura manual. Esto resuelve el caso real donde el coordinador decide seguir atendiendo durante una pausa o extender la atencion por necesidad academica.

Campo	Significado
lvMIni / lvMFin	Inicio y fin del turno de manana de lunes a viernes.
lvTIni / lvTFin	Inicio y fin del turno de tarde de lunes a viernes.
sabIni / sabFin	Horario de sabado.
domActivo	Indica si domingo esta habilitado.
habilitadoManual	Si es true, permite solicitudes aunque el horario normal indique cerrado.

15. Riesgos y Mejoras Futuras

- Para produccion real, separar MySQL en instancias independientes o usar Cloud SQL por servicio.
- Usar HTTPS permanente para el backend, porque Firebase Hosting trabaja sobre HTTPS.
- Mover claves JWT y credenciales a variables seguras o Secret Manager.
- Agregar Swagger/OpenAPI y pruebas automatizadas de integracion.
- Agregar monitoreo con Prometheus/Grafana o Cloud Monitoring si se mantiene en nube.

16. Guion Corto Para Exposicion

1. Presentar el problema: EPISC necesita gestionar certificados de talleres tecnicos con seguimiento por etapas.
2. Explicar actores: alumno registra, coordinador revisa y controla horario, secretaria emite certificados.
3. Mostrar arquitectura: frontend, API Gateway, microservicios y MySQL con schemas separados.
4. Justificar microservicios: separacion por dominio, mantenimiento y despliegue con Docker.
5. Demostrar login, registro de solicitud, cambio de horario y consulta de base de datos.
6. Cerrar con limitaciones: para produccion se recomienda base de datos fisica por servicio y HTTPS permanente.

17. Respuestas Rapidas Para Preguntas Tecnicas

Pregunta probable	Respuesta sugerida
Por que usan una sola base de datos?	Por tiempo y costo academico. La separacion se mantiene por schemas independientes, pero en produccion cada servicio podria migrar a su propia instancia.
Donde esta la seguridad?	El login genera JWT en ms-auth y el API Gateway valida ese token antes de permitir acceso a rutas protegidas.
Como se comunican los servicios?	Mediante APIs REST internas; por ejemplo ms-certificados consulta datos de solicitudes antes de emitir.
Como se despliega?	Docker Compose levanta cada microservicio como contenedor y MySQL

	como contenedor separado.
Que pasa si el coordinador cambia horario?	ms-solicitudes guarda el horario en MySQL; el frontend y la validacion de solicitudes consultan ese estado actualizado.